



Rapport Progweb

Navdeep Singh
Ayman Khemakhem

1 Phase Dev : création du service web

Pour la première étape du projet, nous avons développé deux services web simple en Java avec Spring Boot. Le premier affiche Hello World et le deuxième expose une page de connexion à l'URL `/login`, connectée à une base PostgreSQL, et permet de tester l'authentification d'utilisateurs via une interface web. La base est initialisée automatiquement avec un utilisateur de test (`admin/admin`) grâce à un script SQL exécuté au démarrage.

2 Phase Ops : conteneurisation et publication Docker

Une fois les service fonctionnels en local, nous avons construit une image Docker de chacun de ces services. Ces image incluent toutes les dépendances nécessaires et peuvent être exécutées sur n'importe quelle machine disposant de Docker. Après avoir vérifié le bon fonctionnement des conteneurs localement, nous avons publié les images sur Docker Hub (<https://hub.docker.com/r/navdeep00/service2>) (https://hub.docker.com/r/atiyah/service_login) afin de les rendre accessible pour le déploiement sur Kubernetes.

3 Déploiement Kubernetes

Pour déployer le service sur un cluster Kubernetes (Minikube), nous avons écrit deux fichiers YAML principaux :

- **Déploiement** (`services-deployment.yml`) : définit le pod pour l'application, le pod pour la base PostgreSQL (avec volume persistant et script d'initialisation), ainsi que le pod du second service.
- **Service** (`services.yml`) : expose chaque pod via un service Kubernetes de type `ClusterIP`, permettant la communication interne entre les pods (par exemple, l'application accède à PostgreSQL via le nom DNS `postgres`).

Après application de ces fichiers avec le script `start.sh`, chaque composant fonctionne dans son propre pod et la base de données est accessible par l'application.

4 Ajout d'une gateway Ingress

Pour permettre l'accès externe aux services du cluster, nous avons configuré une ressource **Ingress** (`ingress.yml`). L'Ingress agit comme une passerelle HTTP unique, redirigeant le trafic vers le bon service en fonction du nom d'hôte utilisé :

- Les requêtes vers `service-login.local` sont routées vers le service d'authentification.
- Les requêtes vers `service2.local` sont routées vers le second service.

L’Ingress simplifie ainsi la gestion des accès au cluster et centralise le routage, tout en n’exposant qu’une seule adresse IP publique. Cette configuration est compatible avec l’utilisation d’un Ingress Controller (comme NGINX) ou d’un service mesh (Istio) pour des scénarios plus avancés.

5 Tests et validation

Après déploiement, nous avons vérifié le bon fonctionnement de chaque service en accédant aux URLs prévues via l’Ingress. Des captures d’écran du bon fonctionnement du projet sont disponibles dans le readme du projet github (<https://github.com/Atiyah21/Progweb-projet>).

6 Conclusion

Ce projet illustre la chaîne complète de développement et de déploiement d’un microservice web sur Kubernetes :

- Développement et test local du service web
- Conteneurisation et publication sur Docker Hub
- Déploiement multi-conteneurs sur Kubernetes avec base de données PostgreSQL
- Exposition des services via une gateway Ingress